

Lecture 26 — Solid State Drives and File Systems

Jeff Zarnett

jzarnett@uwaterloo.ca

Department of Electrical and Computer Engineering
University of Waterloo

September 7, 2026

Just as the previous topic started off with talking about the physical construction of a magnetic hard drive, let's start by talking about how solid state drives work.

You might have held a dead hard drive in your hands.
I don't have a dead SSD to show you.

... It is also less interesting.

Once again, solid state drives are called that because they have no moving parts.

That means the question is not the scheduling algorithm for reads and writes; it's a uniform access time device.

And yet, SSDs have some nuances to them that make this more interesting than “hard drives, but faster and uniform”.

SSDs are typically built using NAND Flash memory.

The details of that are interesting, but perhaps too low-level for us to think about.

The important thing that matters for the discussion here, however, is that SSDs are write-limited devices.

They have a maximum number of write cycles and after that, they're finished.
Should I have said cooked? They're cooked.

Nothing is forever and things break. But maximum writes?

Sorry, your CPU has reached its instruction execution maximum and cannot do any more, guess you have to buy a new one.

Wait. Printers from predatory companies do this and I *hate* it.



Less infuriating example: Lithium-Ion batteries have a cycle count.

Why do the NAND flash chips break down with writing?

It's the physics of the thing: the high field and current stress from writing to the chip breaks down the oxide which eventually renders a given cell useless.

Under normal use, you will probably never encounter this.

It's typically something like 10 000 cycles for an individual cell.

Lots of files on the laptop are written once or rarely.

We learned about the swap file earlier: the swap file is read and written a lot!
10 000 writes to the swap file can happen in a day, easily, so then what?

This has led us to something important: when the SSD writes to the same file, that data might not be in the same place(s) on disk anymore.

In fact, even if we wanted to, it would be slow: to write data to a particular block, we have to erase what's there and then write the new block.

But erasing is slow.

So the solution is to write to a new location and then we can go back and erase the old location at a later date.

This has some similarities with page faults!

So physical locations move! Implications:

- 1 There has to be indirection.
- 2 There has to be wear-levelling.
- 3 There has to be garbage collection.
- 4 There are additional writes that are overhead.

1. The level of indirection is called **FTL (Flash Translation Layer)**.
2. Wear-levelling moves things around periodically.
3. Garbage collection: clean up things no longer referenced.
4. Write amplification: how much additional writing takes place.

“Block” is used to refer to a unit of physical storage and does not necessarily have the same size as a page.

Simplest FTL approach: directly mapped.

A read of logical page N is mapped directly to a physical page N and a write is done by reading the whole block with the page in it, erase it, and then overwrite.

We know this is bad: slow to erase.

The current approach used by SSDs is called **log-structured**.

When we write a logical block N , it just gets added to the next free space in the currently-being-written block.

The mapping has to be stored in stable storage.

For every 1 TB of storage where pages are 4 KB, it costs 1 GB of storage just for the mapping table.

This approach does lead to garbage, in the sense of garbage collection mentioned previously.

If a whole block is no longer valid, just erase it.
But what if part of it is still needed?

Can't erase the whole thing if it's partially in-use.

And yet we don't want to waste a ton of space because there's one valid page.

The solution to that is a sort of compaction: if there is garbage in this block, write the valid sections into a new location and then the whole block can be erased.

All of this discussion might have led to the idea that this is all internal to the SSD device itself, and of little concern to the operating system.

That's not quite true, because of the existence of the TRIM command.

Its purpose is for the operating system to inform the SSD what data is no longer needed and can be cleaned up.

The command is needed because many file systems work similarly to C's approach to deallocation of memory.

When a file is deleted, it is just marked as no-longer in use but the actual data is not explicitly deleted/cleared.

So the metadata about the files is updated, but the SSD has never been informed that the data of that file on the disk is garbage now.

This didn't matter for HDDs because if the OS knows that block 9876 is free, it can directly issue a write to that location.

The HDD does not track or care about which parts of the storage medium are used.

When to issue the command?

Design decision!

- Immediately on file delete
- Batch on request
- Periodic

This is synchronous and can cause latency spikes.

Implementing the Interface

Now it is time to go behind the scenes of how the file system lives up to the interface.

The implementation is somewhat complicated, and to keep the size of the problem manageable we will worry about storing files on hard disks.



Hard disks themselves make a good choice for this: they are sufficiently large and sufficiently cheap, to start with, to store the data that we want to store.

We can read an arbitrary part of the disk (unlike a tape).

Finally, we can write to the same part of disk as many times as we want.

Recall also that disks operate on blocks which, in their physical representation, comprise one or more sectors.

- The File System
- I/O Control
- Basic File System
- File Organization Module
- Logical File System

There are a million different file systems (UFS, HFS+, ZFS, NTFS, ext3, FAT32...)

They are all significantly different, but there are some general principles.

A file system will need to keep track of the total number of blocks, the number and locations of free blocks, the directory structure, and the files themselves.

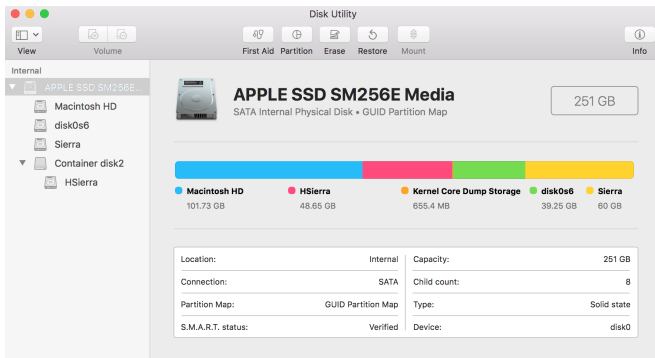
On at least one disk somewhere in the system, there will need to be some information about how to boot up the operating system.

Not every disk has the operating system on it, but if there is an OS the boot loader is usually put in the first block.

When the power button is pressed on the case, the BIOS starts up and transfers control to whatever is found at that first block.

Disks may be split, logically, into several different areas, or **partitions**.

There will be a partition table (sometimes called superblock/master file table).



In the Windows world we often see the whole disk is in one logical partition (the C: drive, for example, taking the whole primary disk).

In Linux we often see the disk divided up to have partitions for different things: temporary/swap directory, home directories, boot partition, et cetera...

There are several structures that are likely to be in memory for performance reasons:

- 1 Mount Table**
- 2 Cache**
- 3 Global Open File Table**
- 4 Process Open File Table**
- 5 Buffers**

Creating a new file is the job of the logical file system.

Allocation of a new FCB (or re-use of an existing free FCB).

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

If a user (application) actually wants to make use of a file: open system call.

Open operates on names, the user comprehensible version.

The file system must then check the global open file table to see if the file is already open somewhere in the system.

If that file is opened for exclusive access, the open routine returns with an error.

If non-exclusive access, then there is no need to search for the file or retrieve it by directory; we just make another reference in the process open file table.

If the file is not already open, then it needs to be retrieved.

Once the file is found, the FCB is copied into the global open file table and the appropriate reference is added to the process table.

The process open file table can contain some additional information, like the next section to read or write, the access mode when the file is open, and so on.

The open system call returns a pointer to the file table and this is the route through which the application performs all file operations.

In UNIX this reference is called a **file descriptor**; in Windows a **file handle**.

The opposite operation to opening a file is obviously to close it.

When a process closes a file, the entry from the open file table can be removed.

If this is the last reference to the file a process has, then the file can also be removed from the global open file table.

Metadata may be updated on close.

Metadata Example: Searching with Spotlight

An example from Mac OS X/macOS: the Spotlight system-wide desktop search.

It was a revelation compared to the previous search options that were slow and did not necessarily include the file content.



Metadata Example: Searching with Spotlight

Spotlight runs queries quickly over the metadata of the system.

Spotlight examines each file on creation and modification and uses it to update the metadata related to that file.

Those items are then added to the Spotlight index which can be queried rapidly.

This is a specific case of preparing the data in advance.

Another example is in exporting data to Excel.

To export this data, there are two possible ways to do it.

One is to examine and process the data for the Excel file at the time of request.

The other is to maintain that data separately, and when the Excel report is asked for, take that data and put it in the Excel file.

Part of the difficulty with maintaining metadata is when to update it.

The longer the time between metadata updates, the higher the chance that there will be a user request that gets back out-of-date data.

The faster the metadata updates, the more time and processing power is spent creating and updating this metadata.

Opening and Reading from a File

